Conditional hardness of algebraic-series truncation via integer factorization and ultimate factorial complexity

Pranjal Dutta, Gorav Jindal, Anurag Pandey, Amit Sinhababu

doi:10.4230/LIPIcs.CCC.2021.25; CC BY4.0

Editorial note. Original authors: Pranjal Dutta, Gorav Jindal, Anurag Pandey, Amit Sinhababu. Official CCC2021 article25, DOI10.4230/LIPIcs.CCC.2021.25.

Editorial note. License: https://creativecommons.org/licenses/by/4.0/

Editorial note. Faithful editable TeX transcription of formal Theorems32/35, complete author Sections6.1/6.2 proofs, full integral-series Theorem77 proof and AppendixI Algorithm1. Original constant-free/nonuniform/fixed-k/all-i/large-d conditions, author expressions and complete published bibliography retained without mathematical correction or new proofs. Only official CC BY4.0 version used; sketch-only permanent consequence excluded.

Allowed standard prerequisites (editorial).

- Binomial series and integer divisibility. Exact author binomial-series formula, Fact74 and Legendre Theorem76 retained.

Author integer-complexity and integrality context

Definition 11 (Straight Line Program). An SLP (straight line program) P (for computing an integer) of length (or size) n is a sequence of integers $a_0, \ldots, a_n$ with $a_0 = 1$ and $a_k = a_i \circ a_j$ with $i, j < k$ for $\circ \in \{+, -, \times\}$. We say that the SLP P computes the integer a_n. For an integer N, we define the straight line complexity $\tau(N)$ of N to be the length of the smallest SLP computing N.

Constant-free circuit complexity. In the same spirit, one can define constant-free circuit complexity of polynomials where the given constants belong to the set $\{-1, 0, 1\}$.[1] We denote, $\tau(f)$ as the size of the minimal constant-free circuit computing f. Trivially, $L(f) \leq \tau(f)$.

[1] To use 2^n in the circuit, one has to build up a circuit for 2^n, of size $\log n$, from 1; whereas in the usual sense of circuit size, constants are free. Thus, $f_d := 2^{2^d} x^d$ has $O(\log d)$-size circuit but requires $\Omega(d)$-size constant-free circuit.

Definition 28 (Easy sequence). A sequence $(a_n)_n$ of integers is said to be "easy to compute" if there exists a polynomial p such that straight line complexity of a_n, i.e. $\tau(a_n) \leq p(\log n)$, for $n \geq 1$. If a sequence is not easy to compute, it is said to be hard. In fact, for most numbers N, one can show that $\tau(N) \geq \log N / \log\log N$ ("close" to the trivial upper bound) [14,35]. It is believed that $(d!)$ is hard to compute. In fact, its hardness is deeply connected to the infamous integer factoring problem. [42] showed that $d!$ being easy to compute will imply factoring is easy in the non-uniform setting.[2]

Definition 33 (Ultimately easy). A sequence of integers (a_n) is ultimately easy if there exists another sequence (b_n) such that $\tau(a_n b_n) \leq \text{poly}(\log n)$ for all large enough n.

Definition 34 (Ultimate complexity). Define the ultimate complexity of an integer n as the minimum τ-complexity of its multiple, i.e. $\tau_1(n) = \min_{b \in \mathbb{Z}\setminus\{0\}} \tau(b \cdot n)$. It is clear that Definition 33 can be stated wrt τ_1. We remark that $\tau_1(n_1 n_2) \leq \tau_1(n_1) + \tau_1(n_2) + 1$, for any $n_1, n_2 \in \mathbb{Z}$.

Binomial series. For rational n, $(x+a)^n = \sum_{k \geq 0} \binom{n}{k} x^k a^{n-k}$, where $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$. A formal power series is $A = \sum_{i \geq 0} A_i x^i$; its degree-$d$ truncation is $\text{trunc}(A, d) = \sum_{0 \leq i \leq d} A_i x^i$.

Fact 74 (Folklore). Product of any k consecutive positive integers is divisible by $k!$.

Definition 75 (p-adic valuation). Let p be a prime and $n \in \mathbb{Z}$. We denote p-adic valuation of n as $\nu_p(n)$ to be the highest exponent such that $p^{\nu_p(n)} \mid n$. Formally, $\nu_p : \mathbb{Z} \longrightarrow \mathbb{N}$ defined by $\nu_p(n) = \max\{v \in \mathbb{N} : p^v \mid n\}$. Note that, by definition, $\nu_p(\text{rad}(n)) = 1$ if $p \mid n$, and 0 otherwise.

Theorem 76 (Legendre's formula). For a prime p and $n \in \mathbb{N}$, $\nu_p(n) = \sum_{j=1}^{\infty} \lfloor n/p^j \rfloor$.

Theorem 77 (Restatement of Theorem 31, Integral power series). Let $k \in \mathbb{N}$. Define $f_k(x) := (1 + k^2 x)^{1/k}$. Then, $f_k(x) \in \mathbb{Z}[[x]]$.

Proof. By binomial expansion, $f_k \in \mathbb{Q}[[x]]$. Let $f_k(x) = \sum_{d \geq 0} a_d x^d$. We'll prove by strong induction that indeed the coefficients are integers. Obviously $a_0 = 1$, and assume that for $m \in \mathbb{N}$ we have proved that $a_\ell \in \mathbb{Z}$ for $0 \leq \ell < m$. The coefficient at x^m in $(\sum_{d=0}^{\infty} a_d x^d)^k = (1 + \sum_{d=1}^{\infty} a_d x^d)^k$ is equal to $k \cdot a_m$ plus a bunch of terms that we know are integer by the induction hypothesis; hence $k \cdot a_m = b \in \mathbb{Z}$. But by the binomial series formula we have

$$a_m = k^{2m} \binom{1/k}{m} = \frac{k^{2m} \prod_{j=0}^{m-1} (1/k - j)}{m!} = \frac{k^m \prod_{j=0}^{m-1} (1 - kj)}{m!}.$$

[2]However, this result does not imply that natural numbers can be factored in polynomial time in the Turing-Machine model, as the numbers used can be poly(n)-bits.

It suffices to prove that $k \mid b$. If we can show that $\nu_p(b) \geq \nu_p(k)$ for every prime p dividing k, this would certainly imply that $k \mid b$. So, fix a prime $p \mid k$. Note that

$$b = k \cdot a_m = X/m!, \qquad X := k^{m+1} \prod_{j=0}^{m-1} (1 - kj).$$

As, $p \mid k$, we must have $\prod_{j=0}^{m-1}(1 - kj) \equiv 1 \bmod p$. Thus, $\nu_p(X) = \nu_p(k^{m+1}) = (m+1)\nu_p(k)$. And by Theorem 76, $\nu_p(m!) = \sum_{j=1}^{\infty} \lfloor m/p^j \rfloor < \sum_{j=1}^{\infty} m/p^j = m/p - 1 \leq m$. Thus,

$$\nu_p(b) = \nu_p(X) - \nu_p(m!) \geq (m+1)\nu_p(k) - m \geq \nu_p(k),$$

as we wanted. Putting it together gives $a_m \in \mathbb{Z}$ proving the inductive step. Hence, the conclusion follows. □

It suffices to prove the integrality of $(1 + k^2x)^{1/k}$ for the candidate functions $(1 + k^2x)^{i/k}$ with $i < k$, as in the authors' Appendix G. In Section6 the authors work throughout with $(1 + k^2x)^{i/k}$, $i, k \in \mathbb{N}$ and $i < k$; Theorem31 states their integrality.

Formal author conditional algebraic-truncation results

Theorem 32. Let $k \in \mathbb{N}$. If $\tau(\mathrm{trunc}((1 + k^2x)^{i/k}, d)) = O(\log^c d)$ (for some constant c) for all $i \in [k-1]$ then integer factorization (in the non-uniform setting) can be performed in polynomial time.

Theorem 35. Fix $k \in \mathbb{N}$. Suppose, for each $i \in [k-1]$, there exists some constant c such that $\tau(\mathrm{trunc}((1 + k^2x)^{i/k}, d)) = O(\log^c d)$, for large enough d. Then, $(n!)_{n\in\mathbb{N}}$ is ultimately easy.

Complete author Section 6.1 proof

Here, we show that if the truncation of each $(1 + k^2x)^{i/k}$, for $i \in [k-1]$, has small constant-free circuit, then one can factor n in $\mathrm{poly}(\log n)$ time, in the non-uniform setting. This would readily imply the first part of Theorem 8.

Proof of Theorem 32. Let, $(1 + k^2x)^{i/k} = \sum_{d\geq 0} C_{d,i}x^d \in \mathbb{Z}[[x]]$, where the coefficient $C_{d,i}$ of x^d is equal to $\pm k^d(-i)(k-i)(2k-i)\cdots((d-1)k-i)/d!$. We see that the product of all $C_{d,i}$ is equal to:

$$\prod_{i\in[k-1]} C_{d,i} = \pm \frac{k^{(k-1)d}(k-1)!(dk)!}{(d!)^k k^d (kd-1)(kd-2)\cdots(kd-(k-1))}.$$

The assumption $\tau(\mathrm{trunc}((1 + k^2x)^{i/k}, d)) = O(\log^c d)$ implies that $\tau(C_{d,i}) = O(\log^c d)$ (just by subtracting two consecutive truncations and substituting $x =$

1). This further implies that $\tau(\prod_{i\in[k-1]} C_{d,i}) = O(\log^c d)$. Let us define, for any $d \geq 1$,

$$N(d,k) := \frac{k^{(k-2)d}(dk)!}{(d!)^k}.$$

We first argue that $N(d,k) \in \mathbb{N}$. This follows from the fact that $N(d,k) = \prod_{i\in[k-1]} C_{d,i} \cdot (kd-1)\cdots(kd-(k-1))/(k-1)!$, and $(k-1)!$ must divide $(kd-1)\cdots(kd-(k-1))$, by Fact 74. Further, since k is constant, it implies that $\tau(N(d,k)) = O(\log^c d)$ (because the extra term has trivial $O(\log d)$-complexity). Now, we describe how to find a non-trivial factor of a given integer n. We assume that all the primes dividing n are larger than k; otherwise we can remove all the prime factors smaller than $k+1$ (since k is a constant). The idea is to first find a positive integer t such that all the primes dividing n are in the interval $[t+1, tk]$, by using an iterative algorithm; if such a t does not exist we would have already found a non-trivial factor of n (by the algorithm). As an invariant, we maintain an integer m such that all the prime divisors of n are greater than m. We start with $m = k$ and compute $\gcd(N(m,k), n)$ at each iteration. Since all the primes dividing n are greater than m (by assumption), we get that $\gcd(N(m,k), n) = \gcd((mk)!, n)$. If the $\gcd((mk)!, n) \neq 1, n$, we must have already found a non-trivial factor of n and we are done. Otherwise, we can have two cases: either (i) $\gcd((mk)!, n) = 1$, or (ii) $\gcd((mk)!, n) = n$. If $\gcd((mk)!, n) = 1$ then we set $m \leftarrow mk$ and continue (because in this case all the primes dividing n must be greater than mk). Otherwise we have $\gcd((mk)!, n) = n$, and hence, all the primes dividing n are in the interval $[m+1, mk]$ and we stop with $t \leftarrow m$. We know that $t \leq \lceil n/k \rceil$ and this uses at most $\log_k n = \log n$ iterations. So, this step has given us an integer t such that all the primes dividing n are in the interval $[t+1, tk]$, and the time taken is $\text{poly}(\log n)$, due to only $\log n$ many iterations and each step takes $\text{poly}(\log n)$-time due to the fact that $\tau(N(d,k)) = O(\log^c d)$ implies gcd computation can be done in $\text{poly}(\log n)$ (by euclidean algorithm). Once, we know that all the primes are in an interval of the form $[t+1, tk]$, we now try to reduce the length of it to k so that, we can simply brute force to get a factor of n, otherwise of course our algorithm would already find a factor. The length reduction part is similar to binary search algorithm that we describe below. To find a positive integer s such that all the primes dividing n are in the interval $[sk+1, (s+1)k]$ (Or we find a non-trivial factor of n), again we use an iterative algorithm. As an invariant, we maintain two positive integers a, b such that all the prime divisors of n are in the interval $[ak+1, bk]$. We start with $a = 1, b = t$. Our invariant is trivially true at the start. At each iteration, we set $c = \lceil (a+b)/2 \rceil$ and compute $\gcd(N(c,k), n)$. Since $c \leq t$ and all the prime divisors of n are larger than t, we get that $\gcd(N(c,k), n) = \gcd((ck)!, n)$. Again, we argue in the same way as before. If the gcd is $\neq 1, n$, we have already found a non-trivial factor of n and we are done. Otherwise, we have two cases: either (i) $\gcd((ck)!, n) = 1$, or (ii) $\gcd((ck)!, n) = n$. If $\gcd((ck)!, n) = 1$ then it is clear that all the primes dividing n are in the interval $[ck+1, bk]$ and hence we set $a \leftarrow c, b \leftarrow b$. If $\gcd((ck)!, n) = n$ then they all the primes dividing n are in the interval $[ak+1, ck]$ and hence we set $a \leftarrow a, b \leftarrow c$. This will terminate when $b - a \leq 1$. Hence we find the desired positive integer s. This uses at most $\log t = \log n$ iterations. Now we just need to search for the prime divisors of n in the interval $[sk+1, (s+1)k]$ (an interval of constant length). Now, we brute force to finally find a non-trivial factor of n. Similarly, this step also

takes poly($\log n$) as each gcd computation takes poly($\log n$) time. So, we have successfully found a non-trivial factor of n by the end of this process, repeating this, we can get all the factors in poly($\log n$)-time and we are done. □We also refer to Algorithm 1 in Appendix I, reproduced below.

Complete author Section 6.2 proof

In this section, we show that easiness of truncation of $(1+k^2x)^{i/k}$ shows that a multiple of $n!$ must be easy. Note that, this may not imply that $n!$ is easy, however, from complexity-theoretic point-of-view, it is believed to be hard because of non-trivial implications. Shub & Smale [44] proved: If $n!$ is ultimately hard to compute, then P $\neq$ NP over the field of complex numbers. Here, the computation is over Blum-Shub-Smale (BSS) model and can use complex numbers in the algorithm. In fact, a stronger version (known as τ-conjecture) connects $z(f)$, distinct integer roots of f with $\tau(f)$. Recently, [16] showed that a similar conjecture, in the SOS-model, would in fact imply explicit constructions of rigid matrices & VP $\neq$ VNP. For similar related works, we refer to [25,27].

Proof of Theorem 35. Let, $(1+k^2x)^{i/k} = \sum_{d\geq 0} C_{d,i}x^d \in \mathbb{Z}[[x]]$. From the hypothesis, it follows that there exists c such that $\tau(C_{d,i}) \leq \log^c d$, for each $i \in [k-1]$ (subtract two consecutive terms and substitute $x=1$). Further, from the proof in Section 6.1 (and following the same notation), we know that

$$\prod_{i\in[k-1]} C_{d,i} = \pm \frac{k^{(k-2)d}(k-1)!(dk)!}{(d!)^k(kd-1)(kd-2)\cdots(kd-(k-1))}.$$

Let us define, $a(d,k) := (dk)!/(d!)^k$. Note that, $a(d,k) \in \mathbb{N}$ (it is the multinomial coefficient $\binom{dk}{d,\ldots,d}$). Further, $k^{(k-2)d}a(d,k) = \prod_{i\in[k-1]} C_{d,i}\cdot(kd-1)\cdots(kd-(k-1))/(k-1)!$, and $(k-1)!$ must divide $(kd-1)\cdots(kd-(k-1))$, by Fact 74. As k is constant, each $kd-i$ can be computed in $O(\log d)$-time trivially. Further, $\tau(\prod_{i\in[k-1]} C_{d,i}) \leq O(\log^c d)$. As τ is additive over multiplication, it follows that

$$\tau(k^{(k-2)d}a(d,k)) \leq O(\log^c d) \implies \tau_1(a(d,k)) \leq O(\log^c d).$$

Now we recurse by noticing the following trivial identity that $n! = n!/(\lfloor n/k \rfloor)!)^k \cdot ((\lfloor n/k \rfloor)!)^k$. We know by the above relation on $a(d,k)$ (and replacing $d := \lfloor n/k \rfloor$ for some integer n) that

$$\tau_1\left(\frac{(k\lfloor n/k \rfloor)!}{(\lfloor n/k \rfloor)!^k}\right) \leq O(\log^c n).$$

Further, any integer n can be written as $n = k\lfloor n/k \rfloor + j$ for some $j \leq k-1$. Note that $k\lfloor n/k \rfloor + j$ has complexity at most $\log n$ for each $j \in [k-1]$. So, multiplying $k\lfloor n/k \rfloor + j$ for $j \in [k-1]$, it is straightforward to deduce that

$$\tau_1\left(\frac{n!}{(\lfloor n/k \rfloor!)^k}\right) \leq O(\log^c n). \tag{2}$$

As, $n! = n!/(\lfloor n/k \rfloor !)^k \cdot ((\lfloor n/k \rfloor)!)^k$, and $\tau_1((\lfloor n/k \rfloor !)^k) \leq \tau_1(\lfloor n/k \rfloor !) + O(1)$; use Equation (2):

$$\begin{aligned}\tau_1(n!) &\leq \tau_1(\lfloor n/k \rfloor !) + O(\log^c n) + O(1) \\ &\leq \tau_1(\lfloor n/k^2 \rfloor !) + O(\log^c n) + O(\log^c n) + O(1) \\ &\vdots \\ &\leq \log_k n \cdot O(\log^c n) = O(\log^{c+1} n).\end{aligned}$$

Therefore, $(n!)$ is ultimately easy to compute, as we wanted. □

Author Appendix I, Algorithm 1

Integer factorization assuming the truncations of $(1 + k^2 x)^{i/k}$ being easy for each i. Input: A composite positive integer n. Output: A non-trivial factor of n.

1. Define $N(d, k) := k^{(k-2)d}(dk)!/(d!)^k$.
2. $m \leftarrow k$.
3. **while true do**
4. Compute $\gcd(N(m, k), n)$.
5. **if** $\gcd(N(m, k), n) = 1$ **then**
6. $m \leftarrow mk$.
7. **else if** $\gcd(N(m, k), n) = n$ **then**
8. $t \leftarrow m$. This m is the desired t.
9. **break**
10. **else**
11. **return** $\gcd(N(m, k), n)$. Here $\gcd(N(m, k), n)$ is a non-trivial factor of n.
12. **end if**
13. **end while**. At this step, all the primes dividing n are in the interval $[t + 1, tk]$.
14. $a \leftarrow 1$.
15. $b \leftarrow t$.
16. **while true do**
17. **if** $b - a \leq 1$ **then**
18. $s \leftarrow a$. This a is the desired s.
19. **break**

20. **end if**

21. $c \leftarrow \lceil (a+b)/2 \rceil$.

22. Compute $\gcd(N(c,k),n)$.

23. **if** $\gcd(N(c,k),n) = 1$ **then**

24. $a \leftarrow c$.

25. **else if** $\gcd(N(c,k),n) = n$ **then**

26. $b \leftarrow c$.

27. **else**

28. **return** $\gcd(N(c,k),n)$. Here $\gcd(N(c,k),n)$ is a non-trivial factor of n.

29. **end if**

30. **end while**. At this step, all the primes dividing n are in the interval $[sk+1,(s+1)k]$.

31. **for** $i = sk+1$ **to** $(s+1)k$ **do**

32. **if** i divides n **then**

33. **return** i. Here i is a non-trivial factor of n.

34. **end if**

35. **end for**

References

[1] Alexander Alder. Grenzrang und Grenzkomplexität aus algebraischer und topologischer Sicht. PhD thesis, Zentralstelle der Studentenschaft, 1984.

[2] Eric Allender, Peter Bürgisser, Johan Kjeldgaard-Pedersen, and Peter Miltersen. On the Complexity of Numerical Analysis. SIAM Journal on Computing, 38, January 2006. Preliminary version in the 21st Annual IEEE Conference on Computational Complexity (CCC'06). doi: 10.1109/CCC.2006.30.

[3] Robert Andrews. Algebraic Hardness Versus Randomness in Low Characteristic. In 35th Computational Complexity Conference (CCC 2020), volume 169 of Leibniz International Proceedings in Informatics (LIPIcs), pages 37:1–37:32. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.CCC.2020.37.

[4] Jean Berstel and Christophe Reutenauer. Rational Series and Their Languages. Springer- Verlag, Berlin, Heidelberg, 1988. URL: https://dl.acm.org/doi/book/10.5555/52107.

[5] Markus Bläser and Gorav Jindal. On the Complexity of Symmetric Polynomials. In 10th Innovations in Theoretical Computer Science Conference (ITCS'19), volume 124 of LIPIcs, pages 47:1–47:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi:10.4230/ LIPIcs.ITCS.2019.47.

[6] Alin Bostan, Gilles Christol, and Philippe Dumas. Fast computation of the Nth term of an algebraic series over a finite prime field. In Proceedings of the ACM on International Symposium on Symbolic and Algebraic Computation (ISSAC'16), pages 119–126, 2016. doi: 10.1145/2930889.2930904.

[7] Karl Bringmann, Christian Ikenmeyer, and Jeroen Zuiddam. On Algebraic Branching Programs of Small Width. J. ACM, 65(5):1–29, 2018. (Preliminary version in the 32nd Computational Complexity Conference (CCC'17). doi:10.1145/3209663.

[8] Peter Bürgisser. The complexity of factors of multivariate polynomials. Foundations of Computational Mathematics, 4(4):369–396, 2004. arXiv:1812.06828.

[9] Peter Bürgisser. On defining integers and proving arithmetic circuit lower bounds. Computa- tional Complexity, 18(1):81–103, 2009. doi:10.1007/s00037-009-0260-x.

[10] Peter Bürgisser, Michael Clausen, and Amin Shokrollahi. Algebraic complexity theory, volume 315. Springer Science & Business Media, 2013.

[11] David V Chudnovsky and Gregory V Chudnovsky. On expansion of algebraic functions in power and Puiseux series, I. Journal of Complexity, 2(4):271–294, 1986. URL: https: //www.sciencedirect.com/science/article/pii/0885064X86900063.

[12] Michael Coons. The Transcendence of Series Related to Stern's Diatomic Sequence. Interna- tional Journal of Number Theory, 06, November 2011. doi:10.1142/S1793042110002958.

[13] Michael Coons and Peter Borwein. Transcendence of power series for some number theoretic functions. Proceedings of the American Mathematical Society, 137, July 2008. doi:10.1090/ S0002-9939-08-09737-2.

[14] Wellington De Melo and Benar Fux Svaiter. The cost of computing integers. Proceedings- American Mathematical Society, 124:1377–1378, 1996. URL: https://www.ams.org/journals/ proc/1996-124-05/S0002-9939-96-03173-5/S0002-9939-96-03173-5.pdf.

[15] Richard A. Demillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. Information Processing Letters, 7(4):193–195, 1978. URL: https://www.sciencedirect.com/ science/article/abs/pii/0020019078900674.

[16] Pranjal Dutta. Real tau-Conjecture for sum-of-squares: A unified approach to lower bound and derandomization. In 16th International Computer Science Symposium in Russia (CSR 2021), 2021. URL: https://drive.google.com/file/d/1X8eo9GM4SCNsC2vWjPbUwMX0vff5i2k3/view.

[17] Pranjal Dutta, Nitin Saxena, and Amit Sinhababu. Discovering the roots: Uniform closure results for algebraic classes under factoring. In Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, pages 1152–1165, 2018. URL: https://www.cse.iitk.ac.in/users/nitin/papers/factor-closure.pdf.

[18] Pranjal Dutta, Nitin Saxena, and Thomas Thierauf. A Largish Sum-Of-Squares Implies Circuit Hardness and Derandomization. In 12th Innovations in Theoretical Computer Science Conference (ITCS 2021), volume 185 of Leibniz International Proceedings in Informatics (LIPIcs), pages 23:1–23:21. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2021. URL: 10.4230/LIPIcs.ITCS.2021.23.

[19] Joshua A Grochow et al. Complexity in ideals of polynomials: questions on algebraic complexity of circuits and proofs. Bulletin of EATCS, 2(130), 2020. URL: http://bulletin.eatcs.org/ index.php/beatcs/article/view/607.

[20] Joshua A. Grochow, Ketan D. Mulmuley, and Youming Qiao. Boundaries of VP and VNP. In 43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016), volume 55, pages 34:1–34:14, 2016. URL: https://core.ac.uk/download/pdf/62922137.pdf.

[21] Gorav Jindal. On approximate polynomial identity testing and real root finding. PhD thesis, Saarland University, 2019. doi:10.22028/D291-29880.

[22] Erich Kaltofen. Uniform closure properties of p-computable functions. In Proceedings of the eighteenth annual ACM symposium on Theory of computing, pages 330–337, 1986. doi: 10.1145/12130.12163.

[23] Erich Kaltofen. Single-factor Hensel lifting and its application to the straight-line complexity of certain polynomials. In Proceedings of the 19th annual ACM symposium on Theory of computing (STOC'87), pages 443–452, 1987. doi:10.1145/28395.28443.

[24] Pascal Koiran. Valiant's model and the cost of computing integers. computational complexity, 13(3):131–146, 2005.

[25] Pascal Koiran. Shallow circuits with high-powered inputs. Innovations in Computer Sci- ence (ICS), 2011. URL: https://hal-ens-lyon.archives-ouvertes.fr/ensl-00477023v4/ document.

[26] Pascal Koiran and Sylvain Perifel. Interpolation in Valiant's theory. Computational Complexity, 20(1):1–20, 2011. doi:10.1007/s00037-011-0002-8.

[27] Pascal Koiran, Natacha Portier, Sébastien Tavenas, and Stéphan Thomassé. A τ -Conjecture for Newton Polygons. Foundations of computational mathematics, 15(1):185–197, 2015. doi: 10.1007/s10208-014-9216-x.

[28] FV Kuhlmann. On convergent power series, 1996. URL: https://www.mathi.uni-heidelberg.de/ roquette/KONVPOTREIHEN.pdf.

[29] Hsiang Kung and Joseph Traub. All Algebraic Functions Can Be Computed Fast. J. ACM,

[30] Dick Lipton and Ken Regan. Factoring and factorials, February 2009. URL: https://rjlipton. wordpress.com/2009/02/23/factoring-and-factorials/.

[31] Richard J Lipton. Polynomials with 0-1 coefficients that are hard to evaluate. SIAM Journal on Computing, 7(1):61–69, 1978. Preliminary version in the 16th Annual Symposium on Foundations of Computer Science (FOCS 1975). URL: https://epubs.siam.org/doi/abs/ 10.1137/0207004?journalCode=smjcat.

[32] Richard J Lipton. Straight-line complexity and integer factorization. In International Algorithmic Number Theory Symposium (ANTS 94), pages 71–79. Springer, 1994. doi: 10.1007/3-540-58691-1_45.

[33] Richard J Lipton and Larry J Stockmeyer. Evaluation of polynomials with super- preconditioning. Journal of Computer and System Sciences, 16(2):124–139, 1978. URL: https://www.sciencedirect.com/science/article/pii/0022000078900417.

[34] Meena Mahajan. Algebraic Complexity Classes. In Perspectives in Computational Complexity, pages 51–75. Springer, 2014. doi:10.1007/978-3-319-05446-9_4.

[35] Carlos Moreira. On asymptotic estimates for arithmetic cost functions. Proceedings of the American Mathematical Society, 125(2):347–353, 1997. URL: https://www.jstor.org/ stable/2161660.

[36] Ketan D Mulmuley. The GCT program toward the P vs. NP problem. Communications of the ACM, 55(6):98–107, 2012. doi:10.1145/2184319.2184341.

[37] Danny Nguyen and Igor Pak. Complexity of short generating functions. In Forum of Mathematics, Sigma, volume 6. Cambridge University Press, 2018. arXiv:1702.08660.

[38] Øystein Ore. Über höhere kongruenzen. Norsk Mat. Forenings Skrifter, 1(7):15, 1922.

[39] Igor Pak. Complexity problems in enumerative combinatorics. In Proceedings of the Inter- national Congress of Mathematicians – Rio de Janeiro 2018. Vol. IV. Invited lectures, pages 3153–3180. World Sci. Publ., Hackensack, NJ, 2018. doi:10.1142/9789813272880_0176.

[40] David A Plaisted. New NP-hard and NP-complete polynomial and integer divisibility problems. Theoretical Computer Science, 31(1-2):125–138, 1984. Preliminary in the 17th Annual Symposium on Foundations of Computer Science (FOCS 1976). URL: https://www.sciencedirect.com/science/article/pii/0304397584901300.

[41] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM (JACM), 27(4):701–717, 1980. doi:10.1145/322217.322225.

[42] Adi Shamir. Factoring numbers in O (logn) arithmetic steps. Information Processing Let- ters, 8(1):28–31, 1979. URL: https://www.sciencedirect.com/science/article/abs/pii/0020019079900875.

[43] Amir Shpilka and Amir Yehudayoff. Arithmetic Circuits: A survey of recent results and open questions. Foundations and Trends® in Theoretical Computer Science, 5(3–4):207–388, 2010. doi:10.1561/0400000039.

[44] Michael Shub and Steve Smale. On the intractability of Hilbert's Nullstellensatz and an algebraic version of "NP $\neq$ P?". Duke Math. J., 81(1):47–54 (1996), 1995. A celebration of John F. Nash, Jr. doi:10.1215/S0012-7094-95-08105-8.

[45] Volker Strassen. Vermeidung von divisionen. Journal für die reine und angewandte Mathematik, 264:184–202, 1973.

[46] L Valiant. Reducibility by algebraic projections in: Logic and algorithmic. In Symposium in honour of Ernst Specker, pages 365–380, 1982.

[47] Leslie G Valiant. Completeness classes in algebra. In Proceedings of the 11th Annual ACM symposium on Theory of computing, pages 249–261. ACM, 1979. doi:10.1145/800135.804419.

[48] Joachim Von Zur Gathen and Jürgen Gerhard. Modern computer algebra. Cambridge university press, 2013.

[49] Wact. Some Accessible Open Problems. Workshop on Algebraic Complexity Theory (WACT 2016). URL: https://www.cs.tau.ac.il/ shpilka/wact2016/concreteOpenProblems/ openprobs.pdf.

[50] Klaus W Wagner. The complexity of combinatorial problems with succinct input representation. Acta informatica, 23(3):325–356, 1986. doi:10.1007/BF00289117.

[51] Richard Zippel. Probabilistic Algorithms for Sparse Polynomials. In Proceedings of the International Symposium on Symbolic and Algebraic Computation, EUROSAM '79, pages 216–226, 1979. doi:10.1007/3-540-09519-5_73.

[52] J.von zur Gathen and V. Strassen. Some polynomials that are hard to compute. Theoretical Computer Science, 11(3):331–335, 1980. URL: http://www.sciencedirect.com/science/ article/pii/0304397580900201.